

Basic Network Commands

NCSC101

Prof. Pranay Kumar Saha

September 22, 2025





What are Network Commands?

Network commands are tools used in command-line interfaces to interact with networks, diagnose issues, and transfer data.

- They help in testing connectivity, transferring files, and remote access.
- Common in Unix/Linux, but available on Windows too.
- Examples: telnet, ftp, sftp, ping.



Importance in Networking

These commands are essential for:

- Troubleshooting network problems.
- Secure and insecure file transfers.
- Remote server access.



What is IP?

The Internet Protocol (IP) is the principal communications protocol in the Internet protocol suite for relaying datagrams across network boundaries.

- Network layer protocol (Layer 3 in OSI model).
- Provides addressing and routing functions.
- Connectionless and best-effort delivery.
- Versions: IPv4 (32-bit addresses) and IPv6 (128-bit addresses).

Example: IP addresses like 192.168.1.1 route packets from source to destination.



IP Addressing and Routing

IP addressing:

- Unique identifiers for devices on a network.
- IPv4: Dotted decimal (e.g., 192.0.2.1).
- IPv6: Hexadecimal (e.g., 2001:db8::1).
- Subnetting divides networks for efficiency.

Routing: Routers use IP headers to forward packets. Discuss headers: source/destination IP, TTL, protocol.



What is ICMP?

Internet Control Message Protocol (ICMP) is a supporting protocol used by network devices to send error messages and operational information.

- Part of the IP suite, but operates over IP.
- Used for diagnostics and error reporting.
- Not for data transfer, but for control messages.
- Examples: Echo Request/Reply (used by ping), Destination Unreachable, Time Exceeded.



ICMP Message Types

Common ICMP messages:

- Type 0: Echo Reply.
- Type 3: Destination Unreachable (e.g., host, port unreachable).
- Type 8: Echo Request.
- Type 11: Time Exceeded (TTL expired).
- Type 5: Redirect (better route available).

Discuss: Ping uses types 8 and 0. Traceroute uses TTL and type 11.



IP Basics

- **IPv4** addresses: dotted decimal (e.g., 192.168.1.10/24)
- **IPv6** addresses: hexadecimal (e.g., 2001:db8::1/64)
- **Subnet/CIDR**: network portion vs host portion (e.g., /24)
- **DNS**: maps names (example.com) to IPs (A/AAAA records)
- **ICMP**: control/diagnostic protocol; used by **ping**
- **TTL**: Time To Live, prevents infinite loops; decremented by hops



Quick IP Inspect Commands

Linux

`ip addr show` # interfaces & IPs

`ip route` # routing table

`cat /etc/resolv.conf` # DNS servers

macOS

`ifconfig`

`route -n get default`

`scutil --dns`

Windows (PowerShell/CMD)

`ipconfig /all`

`route print`

`nslookup example.com`



Ping for Connectivity

One common use is the ping command, which sends ICMP Echo Requests:

```
ping -c 4 google.com
```

- Checks if a host is reachable.
- Measures latency (RTT).
- Detects packet loss.

e.g., 0% packet loss means good connection.



ping Flags (Linux/macOS) —Theory

- **-c <N>**: send N echo requests then stop
- **-i <sec>**: interval between requests (root may go < 0.2s)
- **-s <bytes>**: payload size; tests path MTU/fragmentation
- **-W <sec>**: per-reply timeout (how long to wait for a reply)
- **-t <ttl>** (Linux) / **-m <ttl>** (macOS): set TTL
- **-4/-6**: force IPv4/IPv6
- **-D**: print timestamps; **-q**: quiet output summary only



ping Flags (Linux/macOS) —Examples

```
# Count & timeout
```

```
ping -c 5 -W 2 example.com
```

```
# Timestamped, fast interval
```

```
ping -D -i 0.2 -c 10 example.com
```

```
# Force IPv6
```

```
ping -6 -c 4 ipv6.google.com
```



ping Flags (Windows) —Theory

- **-n <N>**: send N echo requests
- **-w <ms>**: timeout per reply in milliseconds
- **-l <bytes>**: payload size
- **-i <t1>**: set TTL (Windows meaning differs from Linux)
- **-f**: set DF bit (don' t fragment) —MTU testing
- **-4/-6**: force IPv4/IPv6
- **-t**: ping until stopped (Ctrl+C to stop)



ping Flags (Windows) —Examples

REM Send 10 pings with 2s timeout each
`ping -n 10 -w 2000 example.com`

REM Continuous IPv6 ping
`ping -6 -t 100 ipv6.google.com`



Reading ping Output

```
$ ping -c 4 example.com  
--- example.com ping statistics ---  
4 transmitted, 4 received, 0% packet loss  
rtt min/avg/max/mdev = 12.3/15.4/19.8/2.1 ms
```

- **loss** > 0%: congestion, filtering, or target rate-limiting
- **avg RTT**: typical latency; **mdev**: jitter/variability
- **spikes**: intermittent issues or Wi-Fi interference



Introduction to Telnet

Telnet is a protocol and command for remote terminal access.

- Stands for TErminaL NETwork.
- Allows logging into remote systems.
- Uses port 23 by default.
- Insecure: transmits data in plain text.

Example: Use telnet to connect to a mail server.



Telnet Syntax & Common Flags

Basic syntax:

```
telnet [options] [host] [port]
```

Common flags:

- **-a**: Attempt automatic login.
- **-l user**: Specify user for login.
- **-d**: Enable debugging.
- **-8**: 8-bit operation.
- **-E**: Disable escape character.

Prefer modern alternatives for testing: **nc -vz host port** or PowerShell **Test-NetConnection -Port**.



Telnet Example: Testing HTTP

Test a web server on port 80:

```
telnet example.com 80
#in response
Trying 23.215.0.138...
Connected to example.com.
Escape character is '^['.
```

Then type:

```
GET / HTTP/1.1
Host: example.com
```

Press Enter twice.



telnet Port Testing —Examples

```
# HTTP banner test (expect an HTTP response)
```

```
telnet example.com 80
```

```
GET / HTTP/1.1
```

```
Host: example.com
```

```
# SMTP banner test
```

```
telnet mail.example.com 25
```

```
QUIT
```

```
# Alternatives
```

```
Test-NetConnection example.com -Port 443
```



Introduction to FTP

FTP stands for File Transfer Protocol.

- Used for transferring files between client and server.
- Uses ports 20 (data) and 21 (control).
- Supports anonymous access.
- Insecure: Like telnet, plain text.

Example: Downloading files from public FTP servers.



FTP Syntax

Basic syntax to connect:

```
ftp [host]
```

Once connected, common commands:

- **ls**: List files, **cd**: change director, **pwd**: path.
- **get filename**: Download.
- **put filename**: Upload.
- **bye**: Exit.



FTP Example 1: Connecting and Listing

Connect to a public FTP:

```
ftp ftp.example.com
```

Login with 'anonymous' and email as password. Then:

```
ls
```

In class, have students try on a local FTP server.



FTP: Downloading a File

After connecting:

```
get example.txt
```

This downloads the file. Discuss active vs passive mode if issues arise (use 'passive' command).



FTP: Uploading a File

After connecting:

```
put localfile.txt remotefile.txt
```

Explain permissions and binary vs ASCII mode (use 'binary' command).



FTP Security Concerns

FTP transmits credentials in plain text.

- Use FTPS (FTP over SSL) for security.
- Or switch to SFTP.

Teach students about man-in-the-middle attacks.



ftp: Overview

- File Transfer Protocol; control on TCP 21, separate data connection
- Modes: **Active** (server connects back) vs **Passive** (client connects)
- Plain FTP is **not encrypted**; prefer FTPS or SFTP
- We may create temporary FTP server in here
<https://sftpccloud.io/tools/free-ftp-server>



Introduction to SFTP

SFTP is Secure File Transfer Protocol.

- Runs over SSH, uses port 22.
- Encrypted file transfer.
- More secure than FTP.

Requires SSH access to the server.



SFTP vs FTP

FTP	SFTP
Insecure	Secure (SSH)
Ports 20/21	Port 22
Plain text	Encrypted



SFTP vs FTPS

Aspect	SFTP	FTPS
Base Protocol	SSH	FTP with SSL/TLS
Ports	22	21/20 (explicit) or 990 (implicit)
Encryption	Integrated in SSH	Added via SSL/TLS
Authentication	SSH keys, passwords	Certificates, passwords
Features	File management operations	Similar to FTP, may need firewall tweaks



SFTP Syntax and Common Flags

Basic syntax:

```
sftp [options] user@host
```

Common flags:

- -P port: Specify port (default 22).
- -i identity_file: Use private key for authentication.
- -b batchfile: Batch mode for non-interactive.
- -v: Verbose mode.
- -C: Enable compression.

Use -i for key-based auth to avoid passwords. In class, demo with -v to show connection details.



SFTP Example 1: Connecting and Listing

Connect:

```
sftp user@example.com
```

Then:

```
ls
```

Enter password when prompted. Show encryption in action.



SFTP Example 2: Downloading

After connecting:

```
get remote/file.txt local/file.txt
```

Copy directory you need to use recursive downloads with -r.



SFTP Example 3: Uploading

After connecting:

```
put local/file.txt remote/file.txt
```



sftp: Overview

- SSH File Transfer Protocol over TCP 22 (same transport as SSH)
- Encrypted control + data; supports password and key auth
- Good default for secure file transfers and automation



sftp Flags (Common)

- **-P <port>**: port (uppercase P)
- **-i <key>**: identity file (private key path)
- **-o <k=v>**: SSH config option (e.g., **StrictHostKeyChecking=no**)
- **-b <file>**: batch mode (read commands from file)
- **-r**: recursive (directories)
- **-C**: enable compression
- **-v**: verbose; **-q**: quiet



sftp Flags — Examples

```
# Non-default port + identity key + compression  
sftp -P 2222 -i ~/.ssh/id_ed25519 -C user@example.com
```

```
# Batch mode  
cat > commands.txt <<'EOF'  
cd /upload  
put results.csv  
mput *.png  
exit  
EOF  
sftp -b commands.txt user@example.com
```

```
# Single file pull/push without interactive shell  
sftp user@example.com:/remote/file ./local/file  
sftp ./local/file user@example.com:/remote/
```



Key-Based Authentication for SFTP

```
ssh-keygen -t ed25519 -C "student@example" -f ~/.ssh/id_ed25519
ssh-copy-id -i ~/.ssh/id_ed25519.pub user@example.com
# Windows: use ssh.exe and append the .pub content to server's
authorized_keys
```



sftp Troubleshooting

```
# Increase verbosity
sftp -vvv user@example.com

# Common issues (Linux/macOS permissions)
chmod 700 ~/.ssh
chmod 600 ~/.ssh/authorized_keys
# Server: ensure sshd is running and port open in firewall
```



What is SSH?

- SSH stands for Secure Shell.
- It is a cryptographic network protocol for operating network services securely over an unsecured network.
- Commonly used for remote login to computer systems by users.
- Provides a secure channel over an insecure network in a client-server architecture.



History of SSH

- Developed by Tatu Ylönen in 1995 at Helsinki University of Technology.
- Created as a response to a password-sniffing attack at the university.
- SSH-1 was the first version, but had vulnerabilities.
- SSH-2 was introduced in 1998 with improved security.
- OpenSSH, an open-source version, was released in 1999.



How SSH Works

1. **Connection Establishment:** Client contacts server on port 22.
2. **Key Exchange:** They agree on encryption methods and exchange keys.
3. **Authentication:** User authenticates via password, key, or other methods.
4. **Secure Channel:** Data is encrypted and transmitted securely.



SSH Encryption

SSH uses:

- Symmetric encryption for data (e.g., AES).
- Asymmetric encryption for key exchange (e.g., RSA, DSA).
- Hashing for integrity (e.g., SHA).



SSH vs Telnet

SSH

- Encrypted
- Secure authentication
- Modern standard

Telnet

- Unencrypted
- Plaintext passwords
- Obsolete and insecure



Installing SSH on Linux

For Ubuntu/Debian:

```
sudo apt update  
sudo apt install openssh-server  
sudo systemctl start ssh
```



Basic SSH Commands

- Connect: `ssh user@hostname`
- Specify port: `ssh -p 2222 user@hostname`
- Execute command: `ssh user@hostname 'ls -la'`
- Copy files: `scp file user@hostname:/path`



SSH Key Authentication

1. Generate keys: `ssh-keygen -t rsa`
2. Copy public key: `ssh-copy-id user@hostname`
3. Disable password auth in `/etc/ssh/sshd_config`
4. Restart SSH service.

Benefits: More secure than passwords.



Common Issues

- Connection refused: Check if server is running and port is open.
- Permission denied: Verify credentials or key permissions.
- Host key verification failed: Update known_hosts file.
- Timeout: Network issues or firewall blocking.



What is DNS?

- DNS stands for Domain Name System.
- It is a hierarchical and distributed naming system for computers, services, or other resources in the Internet or other Internet Protocol (IP) networks.
- Translates human-readable domain names (like example.com) to machine-readable IP addresses (like 192.0.2.1).
- Acts as the "phonebook" of the internet.



History of DNS

- Developed in the 1980s by Paul Mockapetris.
- Replaced the earlier HOSTS.TXT file system, which was manually maintained and unscalable.
- RFC 882 and RFC 883 published in 1983, defining the concepts.
- Berkeley Internet Name Domain (BIND) software released in 1984.
- DNS became a critical part of the internet infrastructure by the 1990s.



How DNS Works

1. User enters a domain name in a browser.
2. Local resolver queries a recursive DNS server (e.g., ISP's DNS).
3. Recursive server queries root name servers.
4. Proceeds to TLD (Top-Level Domain) servers (e.g., .com).
5. Finally, authoritative name server for the domain provides the IP.
6. Response is cached for future queries.



DNS Record Types

Common types:

- A/AAAA: IPv4/IPv6 address.
- CNAME: Canonical name (alias).
- MX: Mail exchange.
- TXT: Text records (e.g., for SPF, DKIM).
- NS: Name server.

Example query with dig:

```
dig example.com A
```



DNS Security

- DNSSEC: Adds digital signatures to DNS data for authenticity.
- Protects against cache poisoning and spoofing attacks.
- DoH (DNS over HTTPS) and DoT (DNS over TLS): Encrypt DNS queries for privacy.
- Common vulnerabilities: DDoS on DNS servers, amplification attacks.



What are Web Proxies?

- A web proxy is an intermediary server between a client and the internet.
- It forwards client requests to the web server and returns responses.
- Can be forward proxies (for clients) or reverse proxies (for servers).
- Often used for caching, anonymity, or access control.



Types of Web Proxies

- **Forward Proxy:** Sits in front of clients, handles outgoing requests (e.g., corporate proxies).
- **Reverse Proxy:** Sits in front of web servers, handles incoming requests (e.g., load balancers like Nginx).
- **Transparent Proxy:** Intercepts traffic without client configuration.
- **Anonymous Proxy:** Hides user's IP from the destination.
- **Elite/High Anonymity Proxy:** Also hides the fact that a proxy is being used.



Uses of Web Proxies

- **Anonymity and Privacy:** Masks user's IP address to browse anonymously.
- **Bypassing Restrictions:** Access geo-blocked or censored content.
- **Caching:** Stores frequently accessed content to reduce bandwidth and load times.
- **Load Balancing:** Distributes traffic across multiple servers.
- **Security:** Filters malicious content, adds SSL termination.



More Uses of Web Proxies

- **Content Filtering:** Blocks unwanted sites in organizations or schools.
- **Logging and Monitoring:** Tracks user activity for compliance.
- **Compression:** Reduces data usage by compressing responses.
- **Web Scraping:** Proxies rotate IPs to avoid bans.
- **Testing and Development:** Simulate different locations or networks.



Proxy Configuration Examples

In Bash (environment variables):

```
export http_proxy=http://proxy.example.com:8080  
export https_proxy=https://proxy.example.com:8080  
curl https://example.com
```

In browsers: Configure via settings or extensions like FoxyProxy.



Best Practices

- Use trusted proxies to avoid data leaks.
- Combine with VPNs for enhanced security.
- Regularly update proxy software.
- Monitor for abuse or performance issues.

Risks:

- Man-in-the-middle if proxy is compromised.
- Slower speeds due to extra hop.
- Legal issues with bypassing restrictions.



DNS and Proxies Integration

- Proxies can use DNS for name resolution.
- Smart DNS proxies: Alter DNS responses to bypass geo-blocks without routing all traffic.
- Reverse proxies often involve DNS for domain mapping.
- Example: CDN uses DNS to direct users to nearest proxy server.



Basic Network Commands

Thank You for Listening!